KU LEUVEN

Geavanceerde Computerarchitectuur 1st session

GPU programming, course info, exercise description

Thomas Feys, thomas.feys@kueluven.be
DRAMCO – WaveCore – ESAT

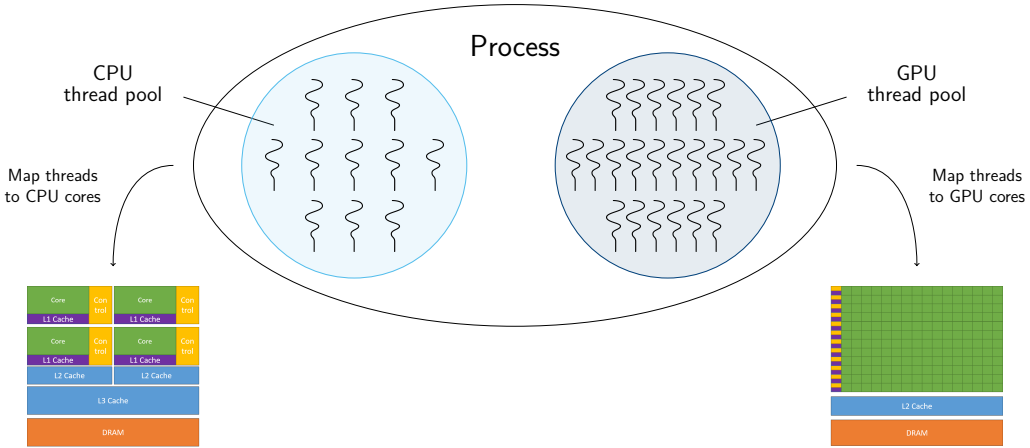
0 Outline

1. Course intro and info
2. A grid of threads
3. Exercise

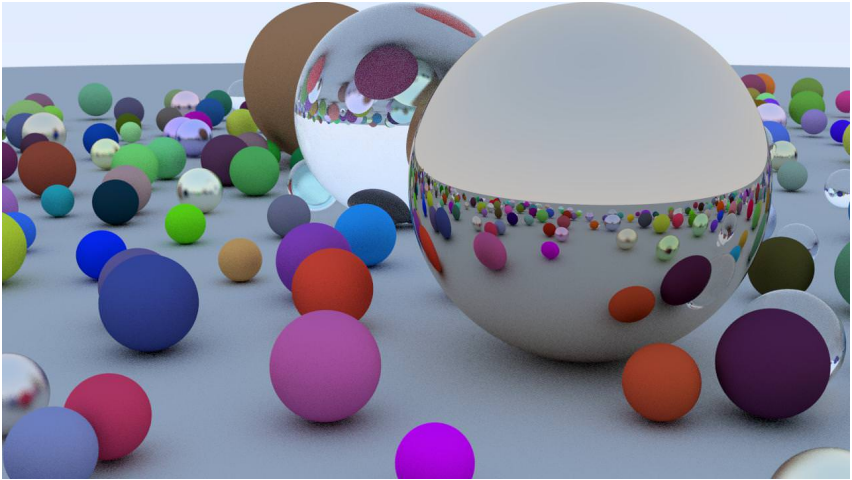
1 Outline

1. Course intro and info
2. A grid of threads
3. Exercise

1 GPUs can execute more threads simultaneously

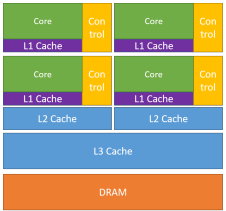


1 Example: parallel processing for ray tracing



1 Relationship between architecture and scope

CPU

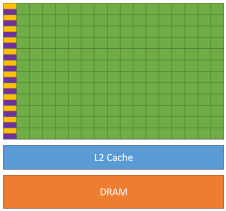


Compilation/execution on the CPU

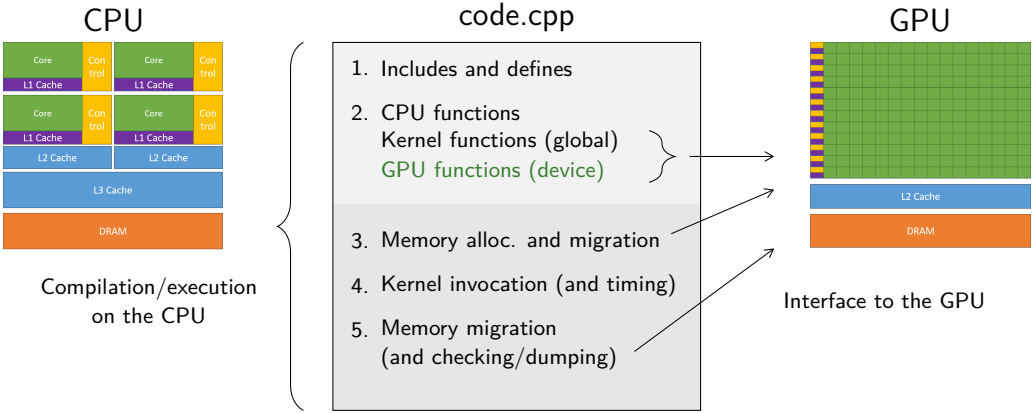
code.cpp

1. Includes and defines
2. CPU functions
   Kernel functions (global)
   GPU functions (device)
3. Memory alloc. and migration
4. Kernel invocation (and timing)
5. Memory migration
   (and checking/dumping)

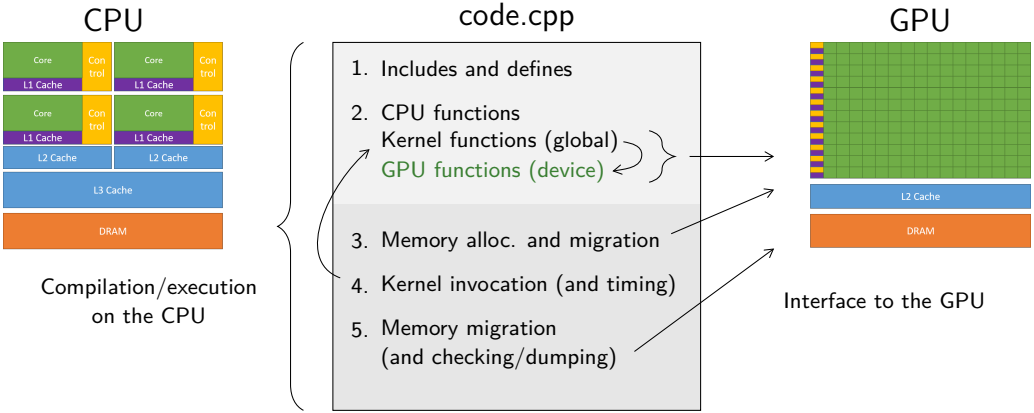
GPU



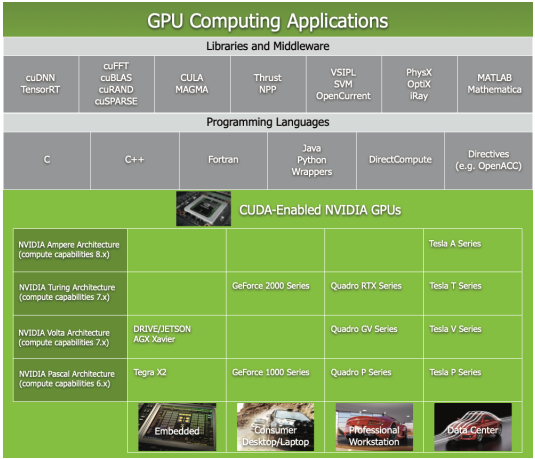
1 Relationship between architecture and scope



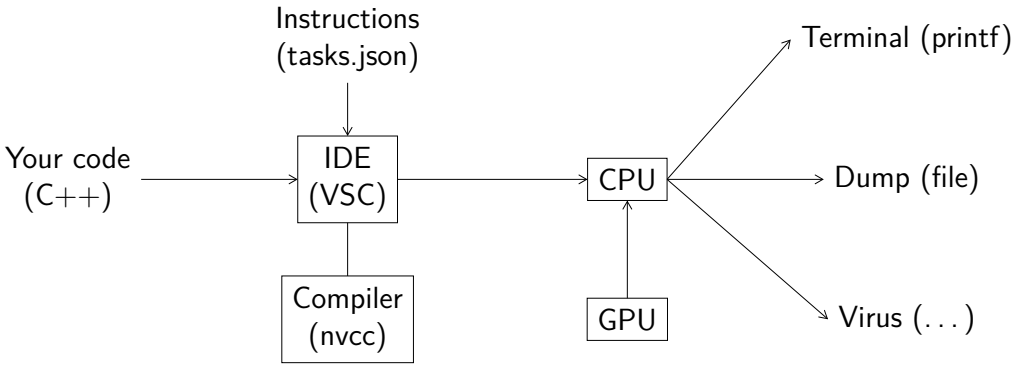
1 Relationship between architecture and scope



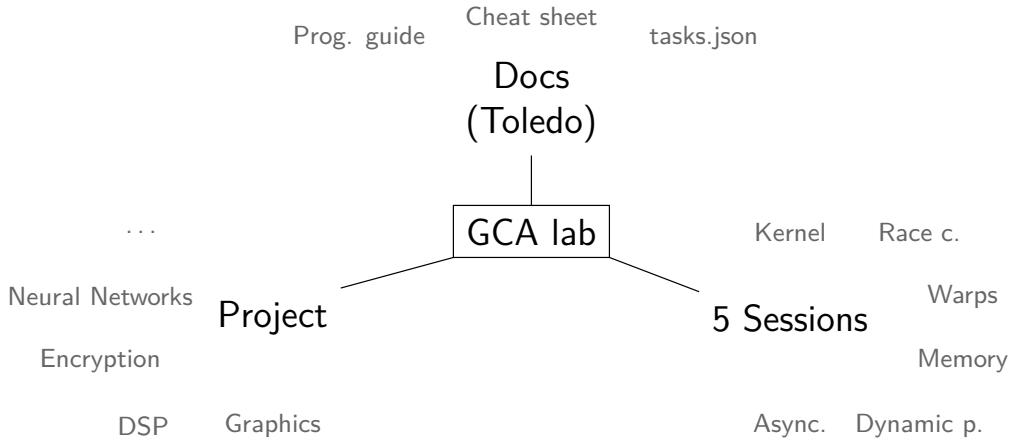
1 CUDA, an API for programming Nvidia GPUs



1 Environment and workflow



1 Organization of the lab and docs



1 Responsibilities and evaluation

Lab sesions, 50 %

- Short report
- Short report
- Short report
- Short report
- Short report

Project, 50 %

- Project report
- Project Presentation
- Code release

1 Report

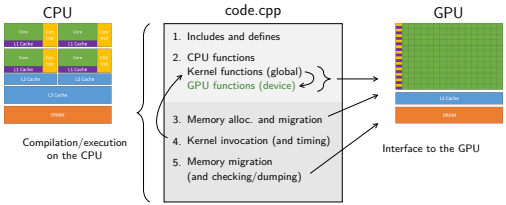
Form of the report:

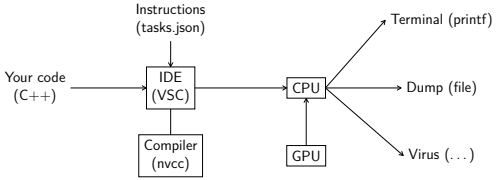
- Short report 1-3 pages.
- Include your code in the appendix!
- List your references!

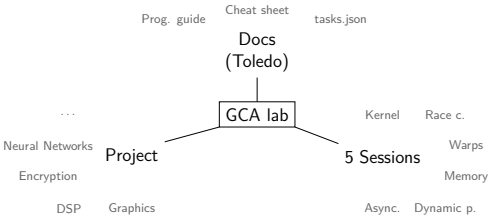
What should be in the report?

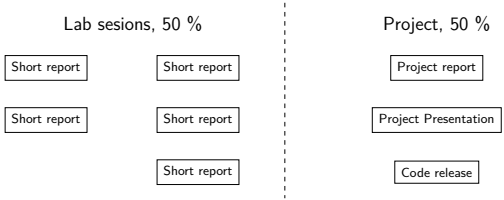
1. Context: describe the problem statement/goal of the lab.
2. Methodology: explain how you tackle the problem and how the code works.
3. Observations/results: from your experiments what do you observe and link this to your expectations/understanding of GPU programming.
4. Conclusion

1 Recap


CPU
code.cpp
GPU
1. Includes and defines
2. CPU functions
Kernel functions (global)
GPU functions (device)
3. Memory alloc. and migration
4. Kernel invocation (and timing)
5. Memory migration
(and checking/dumping)
Compilation/execution
on the CPU
Interface to the GPU


Instructions
(tasks.json)
Your code
(C++)
IDE
(VSC)
CPU
Terminal (printf)
Dump (file)
Virus (. . .)
Compiler
(nvcc)
GPU


Prog. guide
Cheat sheet
tasks.json
Docs
(Toledo)
GCA lab
. . .
Neural Networks
Encryption
DSP
Graphics
Project
5 Sessions
Kernel
Race c.
Warps
Memory
Async.
Dynamic p.

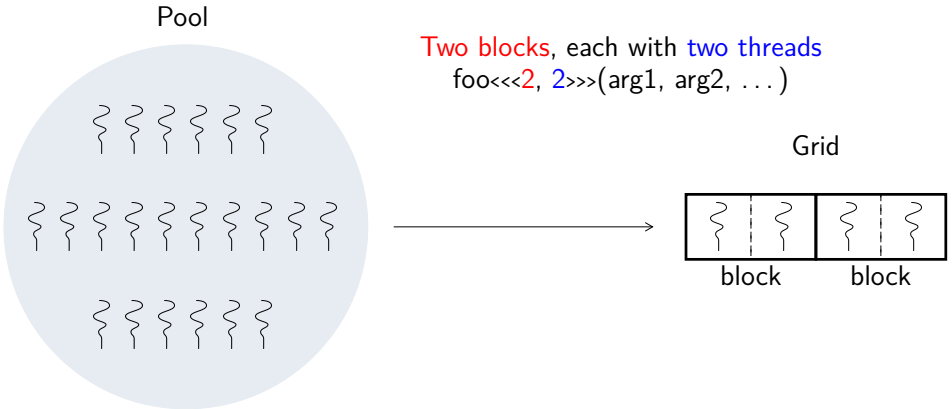

Lab sesions, 50 %
Short report
Short report
Short report
Short report
Short report
Project, 50 %
Project report
Project Presentation
Code release

2 Outline

1. Course intro and info
2. A grid of threads
3. Exercise

2 Threads are organized into blocks, blocks into a grid

- Thread = smallest unit of execution.



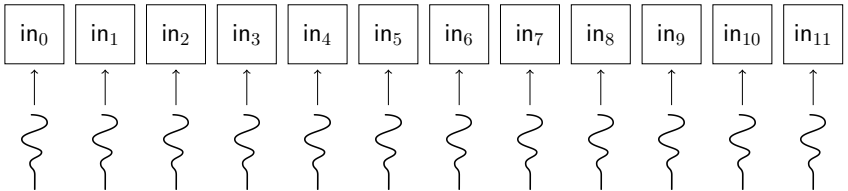
2 Calling a kernel function – example

Goal:

- Process each input element using exactly one thread (not always the case)

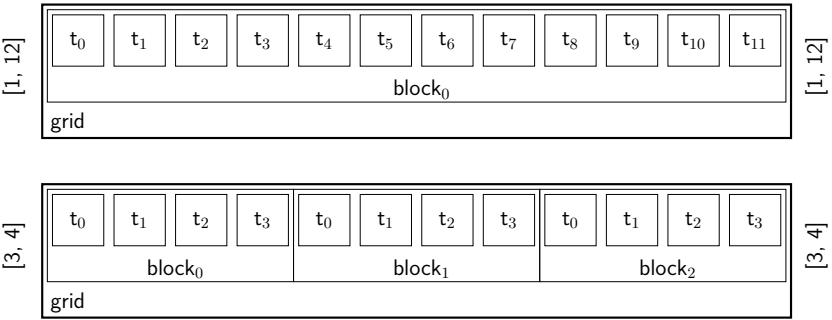
Open questions (OQs):

1. How would you define the block- and grid size? (# threads in block and # blocks in grid)
2. How does a thread know which element it should process?



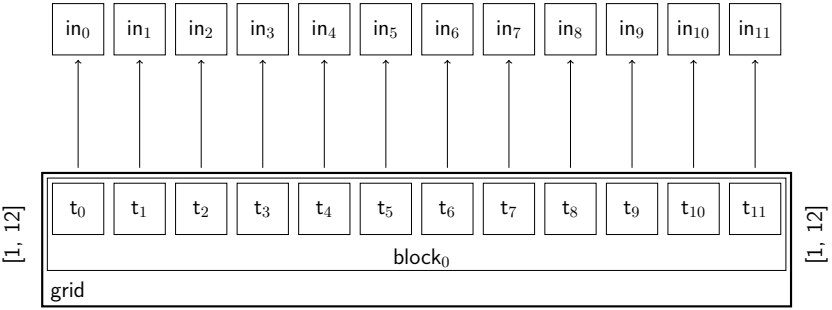
2 OQ_1: You decide how to divide the grid

```
1 foo<<<1, 12>>>(input_data, output_data)
2 foo<<<3, 4>>>(input_data, output_data)
```



2 OQ_2: Importance of correct indexing

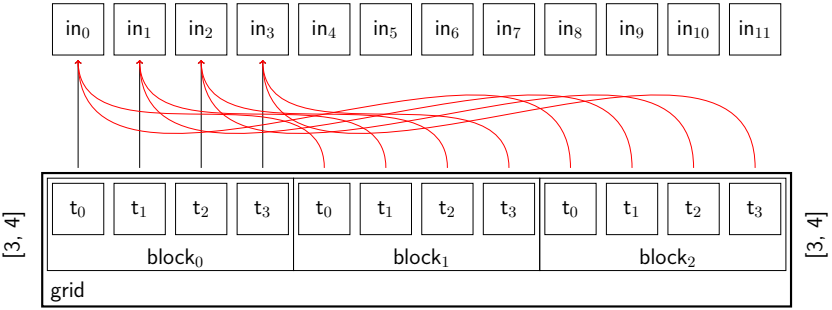
```
1 foo<<<1, 12>>>(input_data, output_data)
```



Good: each thread handles exactly one element.

2 OQ_2: Importance of correct indexing

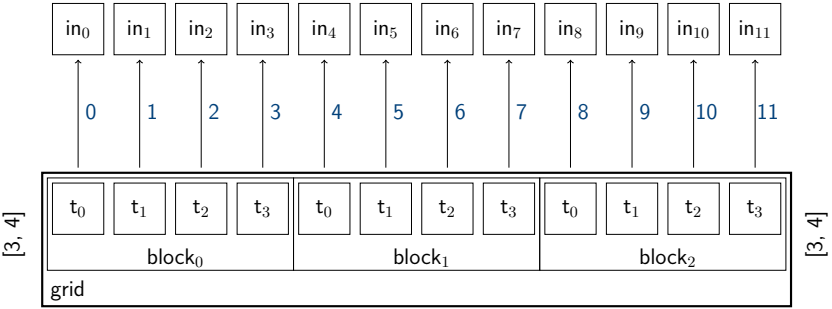
```
1 foo<<<3, 4>>>(input_data, output_data)
```



Problem: overwriting existing- and skipping elements.

2 OQ_2: Correct indexing

- Thread's position within block not enough – relative position only
- Include offset, equal to block size – calculate absolute position within grid



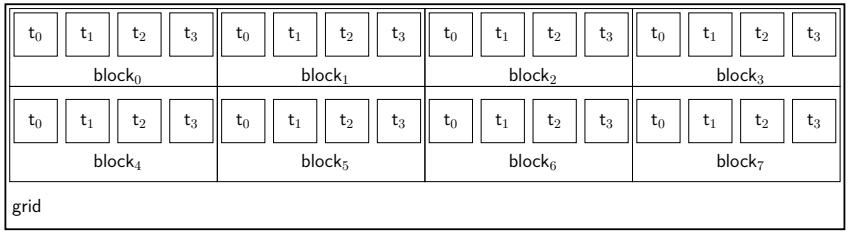
2 Useful built-in functions

- Thread's index within the block `threadIdx.x`
- Block's index within the grid `blockIdx.x`
- Block's size (# threads in block) `blockDim.x`
- Grid's size (# blocks in grid) `gridDim.x`

What is the largest possible block size (max. threads per block)?
See p. 12 of the reference guide.

2 Cuda view vs. physical view

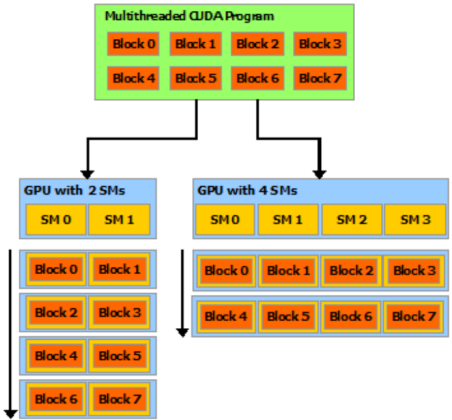
- Cuda view: Grid, blocks, threads




```
1          foo<<<8, 4>>>(input_data, output_data)
2
```

2 Cuda view vs. physical view

- Physical view: GPU (Grid), SMs (a block runs on 1 SM), cores of the SM (1 thread is executed on 1 physical core)



3 Outline

1. Course intro and info
2. A grid of threads
3. Exercise

3 First session's exercise

Problem

- Flip array
- Use multiple blocks with multiple threads

In:	0	1	2	3	4	5	6	7	8
Out:	8	7	6	5	4	3	2	1	0

Goal

- Write and execute parallel code on the GPU
- Migrate memory and verify the results (flipping)

Reports are due by your next lab session.
When finished store your code/results and delete them from the lab PC!!!

3 Extra slide

Images and figures borrowed from:

- https://github.com/mellinoe/veldrid-raytracer
- https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

Resources and documentation:

- cuda programming guide: https://docs.nvidia.com/cuda/cuda-c-programming-guide/
- google is your friend!